call this approach *Do It Right the First Time*. On the other extreme, you might put together some simple code that just solves your obvious problem today, and maybe make it more comprehensive later, as you uncover more special cases. Let's call this approach *Evolutionary Design*. The act of designing code largely consists of striking the right balance between these two approaches.

What do Rails' attribute methods teach us about design? In Rails 1, the code for accessor methods was so simple, you might consider it simplistic. While it was correct and good enough for simple cases, it ignored many nonobvious use cases, and its performance turned out to be problematic in large applications. As the needs of Rails users evolved, the authors of the framework kept working to make it more flexible. This is a great example of Evolutionary Design.

Think back to the optimization in *Rails 2: Focus on Performance*, on page 202. Most attribute accessors, in particular those that are backed by database tables, start their lives as *Ghost Methods (57)*. When you access an attribute for the first time, Active Record takes the opportunity to turn most of those ghosts into *Dynamic Methods (51)*. Some other accessors, such as accessors to calculated fields, never become real methods, and they remain ghosts forever.

This is one of a number of different possible designs. The authors of Active Record had no shortage of alternatives, including the following:

- Never define accessors dynamically, relying on Ghost Methods exclusively.
- Define accessors when you create the object, in the `initialize` method.
- Define accessors only for the attribute that is being accessed, not for the other attributes.
- Always define all accessors for each object, including accessors for calculated fields.
- Define accessors with `define_method` instead of a String of Code.

I don't know about you, but I wouldn't have been able to pick among all of these options just by guessing which ones are faster. How did the authors of Active Record settle on the current design? You can easily imagine them trying a few alternative designs, then profiling their code in a real-life system to discover where the performance bottlenecks were…and *then* optimizing.

The previous example focused on optimizations, but the same principles apply to all aspects of Rails' design. Think about the code in Rails 2 that prevents you from using `method_missing` to call a private method—or the code in Rails 4